C64 Visual Assembler — User Manual

Version 1.4.0

A visual, block-based 6502 assembler for the Commodore 64. Build programs by dragging and dropping instruction blocks, and see the generated assembly and machine code in real time.

Table of Contents

1. Interface Overview

The app is split into three main panels:

Panel	Description
Left — Palette	All available instruction and macro blocks. Search or browse by category.
Center — Program	Your program. Drag blocks here, reorder them, edit operands.
Right — Output	Live ASM view and/or memory monitor output.

2. Block Palette

The palette on the left lists all available blocks grouped by category:

- **Data movement** — LDA, LDX, STA, STX, …
- **Arithmetic** — ADC, SBC, INC, DEC, CMP, …
- **Logic** — AND, ORA, EOR, BIT
- **Jumps & Branches** — JMP, JSR, RTS, BNE, BEQ, …
- **Register operations** — TAX, TAY, INX, DEX, …
- **Shift & Rotate** — ASL, LSR, ROL, ROR
- **Stack** — PHA, PHP, PLA, PLP
- **System** — CLC, SEC, NOP, BRK, …
- **Illegal instructions** — LAX, SAX, DCP, …
- **Structure** — LABEL, COMMENT, GROUP, ENDGROUP
- **Macros** — LOOP, NEXT, PUSH, PULL, TEXT, BYTE, WORD, FILL, ALIGN, STRING, DATA, RAWBYTES, RAWTEXT, PETSCII, INCBIN, SID, INCLUDE, TABLE, MACRO, ENDM, INVOKE, IF, ELSE, ENDIF, SPRITE_INIT, SPRITE_POS, WAIT_RASTER, JOYSTICK, SPRITE_COL

Use the **search box** at the top of the palette to filter by name. Click the **Add selected block** button or drag a block into the program area.

3. Program Area

- **Drag & drop** blocks from the palette, or **reorder** existing blocks by dragging their handle (≡).
- Each block shows its **mnemonic**, **operand field**, and **addressing mode selector** (where applicable).
- Click the **▸ / ▾** toggle to collapse or expand a block.
- Use the **× (delete)** button on a block to remove it.
- **Collapse All** button folds all blocks at once.

Operand input

- For branch/jump instructions (`BNE`, `JMP`, `JSR`, etc.) a **label picker** dropdown appears — click a defined label to insert it.
- Number format follows the **HEX / DEC** toggle in the toolbar (see section 5).

4. ASM View

The right panel shows the generated output in real time.

Output modes

Mode	Description
ASM	6502 assembly source with addresses and labels
Monitor	Hex / byte dump (C64 monitor style)
Both	ASM on top, monitor below
Program	Program settings panel — origin, number format, macro source toggle

Program tab

The **Program** tab contains the settings that affect code generation and output display:

- **Macro source** — when ON, macro definition blocks (MACRO…ENDM) show their source code inline in the ASM view.
- **Show numbers in ASM (HEX / DEC)** — switches all address and value literals in the ASM output between hexadecimal and decimal. This is independent from the per-block HEX / DEC toggle: the per-block toggle controls how you *enter* the operand; this toggle controls how the *output* is displayed.
- **Origin** — the load address of your program (default `$0801`). Supports both `0801` and `$0801` notation. The HEX / DEC toggle next to the input converts the displayed value. All label addresses and the monitor output update immediately.
- **Compile info** — shows a summary of the compiled program (code start address, size, BASIC SYS stub status).

Clicking an ASM line

Click any line in the ASM view to **highlight the corresponding block** in the program area.

---

5. Settings & Toolbar

Control	Description
Number base (HEX / DEC)	Sets the display/input format for operands throughout the UI
Language	Switch between English and Hungarian
Theme	Light / dark mode
CRT retro mode	Toggles a full-screen CRT filter: scanlines, phosphor vignette, flicker, and barrel distortion. State is saved between sessions.
BASIC SYS stub	Prepends a BASIC line that calls SYS to your program's origin
Sample	Load a built-in example program

Control	Description
Zoom in / out	Scale the block UI (affects all block elements)
Save Project	Save the current program as a `.json` project file
Load Project	Load a previously saved project
Save PRG	Export the compiled binary as a `.prg` file
Run in VICE	Compile and launch directly in the VICE emulator
Clear program	Remove all blocks from the program area
Collapse All	Collapse all blocks
About	Version info
What's New	Changelog
Knowledge Base	Reference links (6502 opcodes, C64 KERNAL, memory map, colors)
Check for Update	Open the itch.io page to check for a newer release

6. Addressing Modes

Each 6502 instruction supports one or more addressing modes. The mode selector appears on each block.

Mode	Label	Example	Description
implied	Implied	`NOP`	No operand; the instruction is self-contained
immediate	Immediate	`LDA #$FF`	Inline constant; the assembler adds `#` automatically
zeroPage	Zero page	`LDA $10`	Single byte address in page zero (0–255)
absolute	Absolute	`LDA $0400`	Full 16-bit memory address
relative	Relative/Label	`BNE loop`	For branch instructions; enter a label name or target address
absoluteX	Absolute,X	`LDA $0400,X`	16-bit address + X register offset
absoluteY	Absolute,Y	`LDA $0400,Y`	16-bit address + Y register offset
indirectX	Indirect,X	`LDA ($FB,X)`	Zero page indexed indirect
indirectY	Indirect,Y	`LDA ($FB),Y`	Zero page indirect indexed
indirect	Indirect	`JMP ($0100)`	Indirect; only usable with JMP
zeroPageY	Zero page,Y	`LDX $FB,Y`	Zero page address + Y register offset

7. Standard 6502 Instructions

Data Movement

Mnemonic	Description	Modes
`LDA`	Load Accumulator	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`LDX`	Load X register	immediate, zeroPage, zeroPageY, absolute, absoluteY
`LDY`	Load Y register	immediate, zeroPage, absolute, absoluteX
`STA`	Store Accumulator	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`STX`	Store X register	zeroPage, zeroPageY, absolute
`STY`	Store Y register	zeroPage, absolute

Arithmetic

Mnemonic	Description	Notes
`ADC`	Add with carry	Set carry with `SEC` before use in most cases
`SBC`	Subtract with carry	Set carry with `SEC` before subtraction
`INC`	Increment memory	—
`DEC`	Decrement memory	—
`CMP`	Compare with A	Sets flags; does not modify A
`CPX`	Compare with X	—
`CPY`	Compare with Y	—

Logic

Mnemonic	Description
`AND`	Logical AND with Accumulator
`ORA`	Logical OR with Accumulator
`EOR`	Exclusive OR with Accumulator
`BIT`	Test bits in memory against A (sets N, V, Z flags)

Jumps & Branches

Mnemonic	Description

Mnemonic	Description
JMP	Unconditional jump (absolute or indirect)
JSR	Jump to subroutine (saves return address on stack)
RTS	Return from subroutine
RTI	Return from interrupt
BNE	Branch if Not Equal (Z=0)
BEQ	Branch if Equal (Z=1)
BCC	Branch if Carry Clear (C=0)
BCS	Branch if Carry Set (C=1)
BMI	Branch if Minus (N=1)
BPL	Branch if Plus (N=0)
BVC	Branch if Overflow Clear (V=0)
BVS	Branch if Overflow Set (V=1)

Register Operations

Mnemonic	Description
TAX	Transfer A → X
TAY	Transfer A → Y
TXA	Transfer X → A
TYA	Transfer Y → A
TSX	Transfer Stack pointer → X
TXS	Transfer X → Stack pointer
INX	Increment X
DEX	Decrement X
INY	Increment Y
DEY	Decrement Y

Shift & Rotate

Mnemonic	Description
ASL	Arithmetic Shift Left
LSR	Logical Shift Right

Mnemonic	Description
ROL	Rotate Left through Carry
ROR	Rotate Right through Carry

Stack

Mnemonic	Description
PHA	Push Accumulator onto stack
PHP	Push Processor status onto stack
PLA	Pull Accumulator from stack
PLP	Pull Processor status from stack

System / Flags

Mnemonic	Description
CLC	Clear Carry flag
CLD	Clear Decimal mode
CLI	Clear Interrupt disable
CLV	Clear Overflow flag
SEC	Set Carry flag
SED	Set Decimal mode
SEI	Set Interrupt disable
NOP	No operation
BRK	Force break / software interrupt

Illegal / Undocumented Instructions

These are supported for advanced use. Use with care — behavior may differ between chips.

LAX, SAX, DCP, ISC, SLO, RLA, SRE, RRA, ANC, ALR, ARR, AXS

---

8. Macro Blocks — Reference

Macro blocks generate multiple instructions or data directives automatically. They are found in the **Macros** category of the palette.

LABEL

Creates a named label that can be used as a jump target.

Field	Description
Label name	Identifier used in `JMP`, `JSR`, `BNE`, etc.

Generated ASM:

```
loop:  ; $0820
```

The address is shown as a comment. Labels have **0 byte size**.

COMMENT

Inserts a comment line in the ASM output. No bytes generated.

Generated ASM:

```
; Your comment text here
```

BYTE

Inserts an arbitrary sequence of raw bytes.

Field	Description
Operand	Comma-separated byte values (e.g. `$01, $02, $FF` or `1, 2, 255`)

Generated ASM:

```
    .byte $01, $02, $FF
```

Size: Number of bytes in the list.

WORD

Inserts 16-bit values stored as little-endian LO/HI byte pairs.

Field	Description
Operand	Comma-separated 16-bit values (e.g. `$0400, $C000`)

Generated ASM:

```
    .word $0400, $C000
```

Size: 2 bytes per word.

FILL

Generates a repeated sequence of the same byte value.

Field	Description
Operand	`count,value` — e.g. `256,0` fills 256 bytes with zero

Generated ASM:

```
    .fill 256, $00
```

Size: The count value in bytes.

ALIGN

Advances the program counter to the next boundary of the given size. Inserts padding bytes as needed.

Field	Description
Boundary	Alignment value — e.g. `64` (sprite boundary), `256` (page), `$2000` (bitmap)

Generated ASM:

```
    ; ALIGN 64 → $0840 (12 bytes padding)
```

Size: Dynamic — depends on the current program counter position.

> **Tip:** Use `ALIGN 64` before sprite data, `ALIGN 256` to ensure page-aligned tables.

TEXT

Writes a text string to the C64 screen RAM at a given row/column position using the KERNAL CHROUT routine.

Field	Description
Text	The string to display
X	Column (0–39)
Y	Row (0–24)

Generated ASM:

```
LDA #$48      ; 'H'
JSR $FFD2
LDA #$45      ; 'E'
JSR $FFD2
...
```

Characters are encoded as PETSCII. **Size:** `text.length × 5` bytes (LDA + JSR per character).

STRING

Places a PETSCII-encoded string as raw bytes at a given memory address (no runtime code, deferred data section).

Field	Description
Text	The string to place
Address	Target memory address (e.g. `$C000`)

Generated ASM:

```
.byte $48, $45, $4C, $4C, $4F   ; "HELLO"
```

Placed at the specified address. **Size:** `text.length` bytes.

DATA

Writes raw bytes to a given memory address using LDA/STA instructions at runtime.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address

Generated ASM:

```
LDA #$01
STA $C000
LDA #$02
STA $C001
...
```

Size: `byte_count × 5` bytes (one LDA + one STA per byte).

RAWBYTES

Places raw bytes at a given memory address — no runtime code is generated. The bytes appear in the deferred data section of the output.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address (e.g. `$C000`)

Size in code: 0 bytes. The data is placed at the given address in the output.

> **Use this instead of DATA** when you don't need runtime initialization code and just want bytes at an address.

RAWTEXT

Like RAWBYTES but accepts a text string that is encoded as PETSCII bytes.

Field	Description
Text	String to encode
Address	Target memory address

Size in code: 0 bytes.

PETSCII

Places a text string encoded as PETSCII bytes at a given memory address. No runtime code is generated — the bytes are placed directly at the target address, similar to RAWBYTES.

Each printable ASCII character (codes 32–126) is stored using its standard ASCII value, which corresponds to the PETSCII uppercase range. The result is compatible with KERNAL CHROUT (`$FFD2`).

Field	Description
Text	String to encode as PETSCII bytes
Address	Target memory address (e.g. `$C000`)

Generated ASM:

```
; .petscii "HELLO" -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; H E L L O
```

Size in code: 0 bytes. The data is placed at the target address as a deferred data section (like RAWBYTES).

Encoding rules:

Input	Byte value
Printable ASCII (space, letters, digits, punctuation)	Standard ASCII code (32–126)
Newline	`$0D` (RETURN)
Everything else	`$20` (space)

> **Tip:** Use PETSCII for data that will be output via CHROUT (`$FFD2`). For screen-code strings (different mapping), use the STRING macro instead.

INCBIN

Includes an external binary file and places its content at a given memory address.

Field	Description
File	Browse to select a `.bin`, `.prg`, `.sid`, or `.raw` file
Address	Target load address (e.g. `$C000`)

Generated ASM comment:

```
; INCBIN "music.bin" @ $C000 (2048 bytes)
.byte $01, $02, ...
```

Size in code: 0 bytes (deferred data section). The binary is embedded at the given address.

SID

Loads a SID music file and places its data at the address specified in the SID header. Header metadata (Load/Init/Play addresses, title, author) is extracted automatically.

Field	Description
File	Browse to select a `.sid` file

The block displays:

- **Title / Author** from the SID header
- **Load address** — where the data is placed in memory
- **Init address** — call this with JSR to initialize the music
- **Play address** — call this with JSR on every frame (e.g. in an IRQ handler)

Generated ASM comment:

```
; SID "Commando.sid" @ $1000  Init:$1000  Play:$1003  (8192 bytes)
```

Size in code: 0 bytes. The SID data is placed at the load address.

> **Typical usage:** JSR to Init once, then call Play periodically from an IRQ.

INCLUDE

Includes another Visual Assembler project file and expands its blocks inline at this position. The included blocks are read-only.

Field	Description
File	Browse to select a `.json` Visual Assembler project

Generated ASM:

```
; === INCLUDE "library.json" — 12 block(s) ===
... (expanded blocks follow)
```

> **Tip:** Use INCLUDE to build reusable subroutine libraries that you can share across projects.

TABLE

Defines a lookup table at a fixed memory address with a named label.

Field	Description
Name	Label identifier for the table (e.g. `color_table`)
Address	Fixed address where the table starts (e.g. `$C000`)

Generated ASM:

```
color_table:
```

The program counter jumps to the specified address for subsequent blocks. Place BYTE/WORD/FILL blocks after TABLE to fill the table content.

Size: 0 bytes.

LOOP / NEXT

A counted loop pair using a register as the counter.

LOOP

Field	Description
Register	`X` or `Y` — the counter register
Count	Loop iteration count (hex or decimal, e.g. `0A` = 10)
Label	Auto-generated loop label (e.g. `loop0`)

Generated ASM:

```
    LDX #$0A
loop0:
```

Size: 2 bytes (LD_ opcode + immediate operand).

NEXT

Field	Description
Register	Automatically matched to the LOOP register
Label	Automatically linked to the LOOP label

Generated ASM:

```
    DEX
    BNE loop0
```

Size: 3 bytes (DEX + BNE + branch offset).

Example — clear 10 screen cells:

```
    LDX #$0A
loop0:
    LDA #$20        ; space character
    STA $0400,X
    DEX
    BNE loop0
```

PUSH / PULL

Save and restore registers using the stack.

PUSH

Pushes one or more registers onto the stack. The order is always A → X → Y (innermost first).

Field	Description
Registers	Any combination: A, X, Y, AX, AY, XY, AXY

Generated ASM (example: AX):

```
PHA
TXA
PHA
```

Size: 1 byte for A (PHA), 2 bytes for X or Y (transfer + push).

PULL

Restores registers from the stack in reverse order (Y → X → A).

Field	Description
Registers	Same as PUSH — must match the corresponding PUSH block

Generated ASM (example: AX):

```
PLA
TAX
PLA
```

> **Important:** Always pair PUSH and PULL with the same register set, and in reverse order. PUSH AX must be matched by PULL AX (which internally does Y-first, then X, then A).

MACRO / ENDM / INVOKE

Define and reuse your own named code blocks.

MACRO (definition start)

Field	Description
Name	Identifier for the macro (e.g. clear_screen)

Marks the beginning of a macro definition. All blocks between MACRO and ENDM become the macro body. The definition does **not** generate code where it appears.

Generated ASM:

```
; .MACRO clear_screen
    ... (body blocks)
; .ENDM
```

ENDM (definition end)

Closes the current macro definition. No fields.

INVOKE

Inserts the contents of a named macro at this position.

Field	Description
Macro name	Select from the dropdown of defined macros

Generated ASM:

```
; >>> Invoke: clear_screen
    ... (expanded macro body)
```

The macro body is expanded inline — all addresses and labels are resolved in context.

> **Tip:** Define macros at the top (or bottom) of your program, then INVOKE them wherever needed. Macros can be invoked multiple times.

GROUP / ENDGROUP

Groups a set of blocks into a **named, collapsible section**. GROUP and ENDGROUP are purely visual — they generate **zero bytes** and have no effect on the assembled output.

Field	Description
Group name	Free-text label for the section (e.g. `init`, `game_loop`, `sprite_setup`)

Controls on the GROUP block:

- ▸ / ▾ **toggle** — collapses or expands the entire group. When collapsed, all blocks between GROUP and ENDGROUP are hidden and the GROUP block's own body folds in.
- ⊡ **Expand all** — un-collapses every individually collapsed block inside the group and expands the group itself if needed.
- ⊖ **Select in ASM** — highlights the entire group's code range in the ASM view (from `; ===[ name ]===` to `; ===[/name]===`) and scrolls to it. Switches to the ASM tab automatically if it is not currently visible.

Generated ASM:

```
; ===[ init ]===
    SEI
    LDA #$00
    STA $D020
; ===[/init]===
```

Size: 0 bytes for both GROUP and ENDGROUP.

Example workflow:

1. Add a `GROUP` block, set name to `init`.
2. Add your initialization instructions below it.
3. Add an `ENDGROUP` block to close the section.
4. Click ▸ on the GROUP to collapse the whole section into one line while working on other parts of the program.

> **Note:** Groups do not nest. Placing a second GROUP before an ENDGROUP starts a new group at the same level.

DEFINE / IF / ELSE / ENDIF

Conditional assembly blocks. Add a `DEFINE` block to activate a named symbol — any `IF` block whose condition matches a `DEFINE` present in the program is assembled; its `ELSE` branch (if any) is skipped, and vice versa.

DEFINE

Field	Description
Symbol	One or more comma-separated identifiers to activate (e.g. `DEBUG` or `DEBUG, PAL`)

Generated ASM:

```
; .DEFINE DEBUG
; .DEFINE DEBUG, PAL
```

A single `DEFINE` block can activate multiple symbols at once using a comma-separated list. Place `DEFINE` blocks anywhere in the program (typically at the top). Removing the block deactivates all its symbols instantly.

IF

Field	Description
Condition	Identifier to test (must match a `DEFINE` symbol to be active)

Generated ASM:

```
; .IF DEBUG
```

Blocks between `IF` and `ENDIF` (or `ELSE`) are included or skipped based on whether the condition symbol has a matching `DEFINE` block in the program. Skipped blocks appear as `; [IF skipped] …` comments and generate **zero bytes**.

ELSE

No fields. Marks the alternative branch — assembled when the `IF` condition is *not* active.

Generated ASM:

```
; .ELSE
```

ENDIF

No fields. Closes the conditional block.

Generated ASM:

```
; .ENDIF
```

Size: 0 bytes for all four blocks. Only the content *between* them counts.

Example — debug border flash, release build skips it:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02        ; red border
    STA $D020
; .ELSE
    LDA #$00        ; black (release)
    STA $D020
; .ENDIF
```

Example — multiple symbols in one DEFINE block:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx        ; PAL timing constant
; .ELSE
```

```
    LDA #$xx        ; NTSC timing constant
; .ENDIF
```

Nested `IF` blocks are supported. The innermost condition is evaluated independently; an outer skipped block causes all inner blocks to be skipped regardless of their condition.

CONST

Declares a named constant. The constant is added to the label table and can be referenced as an operand in any instruction block (LDA, STA, JSR, JMP, etc.).

Field	Description
Name	Identifier for the constant (e.g. `SCREEN`)
Value	Numeric value in the selected base (e.g. `0400` in HEX = address $0400)
Format	HEX or DEC — controls how the value is entered and displayed

Generated ASM:

```
; .CONST SCREEN = $0400
```

The constant name appears in the **label picker** dropdown of instruction blocks that support absolute/immediate addressing — simply click it to insert the constant name as the operand.

Size: 0 bytes.

SPRITE_INIT

Initialises a VIC-II sprite in a single block: sets the **data pointer**, **enable bit**, and **colour**.

Field	Description
Sprite #	Sprite number 0–7
Colour	Colour index 0–15 (C64 palette)
Data page	Sprite data address / 64 (e.g. `$21` if data is at `$0840`)

Generated ASM:

```
    LDA #$21
    STA $07F8       ; sprite pointer register ($07F8 + N)
    LDA $D015
    ORA #$01        ; set enable bit for sprite 0
    STA $D015
```

```
    LDA #$07
    STA $D027       ; sprite 0 colour register
```

Size: 18 bytes.

> **Sprite data page:** `data_address / 64`. With the default BASIC SYS stub, an `ALIGN 64` block after `JMP main` places sprite data at `$0840` → page = `$0840 / 64 = 33 = $21`.

SPRITE_POS

Sets a sprite's **static start position** (compile-time constant). Use this for initial placement; for animation use `INC`/`DEC` on the sprite's register directly.

Field	Description
Sprite #	Sprite number 0–7
X	Horizontal position 0–319
Y	Vertical position 0–255

Generated ASM (example: sprite 0, X=152, Y=100):

```
    LDA #$98        ; X low byte
    STA $D000       ; sprite 0 X register
    LDA $D010
    AND #$FE        ; clear X MSB for sprite 0 (X ≤ 255)
    STA $D010
    LDA #$64        ; Y = 100
    STA $D001       ; sprite 0 Y register
```

For X > 255 the macro sets the corresponding bit in `$D010` instead of clearing it.

Size: 18 bytes.

> **Note:** `SPRITE_POS` bakes the position into the code at assemble time (`LDA #$xx`). To move a sprite at runtime use `INC $D000` / `DEC $D000` directly — see the `sprite-macro-demo` sample.

WAIT_RASTER

Waits for a specific VIC-II raster line — entirely **inline**, no JSR and no external label required.

Field	Description
Raster line	Target raster line in hex (e.g. `FF` = line 255)

Generated ASM:

```
wait:
    LDA $D012       ; current raster line
    CMP #$FF        ; target line
    BNE wait        ; loop back (-7 bytes)
```

Size: 7 bytes (the `BNE` offset `$F9` = −7 always points back to the `LDA`).

> **Tip:** Place `WAIT_RASTER` at the top of your game loop to synchronise with the display and prevent sprite tearing.

JOYSTICK

Reads a CIA joystick port and moves a sprite according to the direction pressed. Entirely **inline** — no JSR or label needed.

Field	Description
Port	`1` = port 1 (`$DC01`) or `2` = port 2 (`$DC00`)
Sprite #	Sprite number 0–7 (controls which X/Y register pair is updated)

Generated ASM (port 2, sprite 0):

```
    LDA $DC00       ; read CIA port 2
    LSR             ; bit 0 → carry (Up)
    BCS skip_up     ; carry set = NOT pressed
    DEC $D001       ; Y−1 (move up)
skip_up:
    LSR             ; bit 1 → carry (Down)
    BCS skip_down
    INC $D001       ; Y+1 (move down)
skip_down:
    LSR             ; bit 2 → carry (Left)
    BCS skip_left
    DEC $D000       ; X−1 (move left)
skip_left:
    LSR             ; bit 3 → carry (Right)
    BCS skip_right
    INC $D000       ; X+1 (move right)
skip_right:
```

Joystick bit map (active-LOW — bit = 0 means pressed):

Bit	Direction	CIA register
0	Up	$DC00 (port 2) / $DC01 (port 1)
1	Down	

Bit	Direction	CIA register
2	Left	
3	Right	
4	Fire	(not handled by this macro)

Size: 27 bytes. The `BCS` offset is always `+3` (skips the following 3-byte `DEC`/`INC abs` instruction).

> **Typical usage:** Place inside a `gameloop` label with `WAIT_RASTER` first:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    JMP gameloop
```

SPRITE_COL

Detects sprite collisions using the VIC-II hardware collision registers. Entirely **inline** — no JSR or label needed.

Field	Description
Sprite #	Sprite number 0–7 (which sprite's bit to check)
Collision type	`Sprite-Sprite ($D01E)` — collision with another sprite; `Sprite-Background ($D01F)` — collision with background graphics

Generated ASM (sprite 0, sprite–sprite):

```
    LDA $D01E       ; read sprite-sprite collision register (clears it!)
    AND #$01        ; isolate bit 0 (sprite 0)
                    ; A ≠ 0 → collision occurred
```

Size: 5 bytes.

> **Important:** Reading `$D01E` / `$D01F` **automatically clears the register**. Always read it exactly once per frame — use the result immediately with `BEQ`/`BNE`.

Typical usage:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    SPRITE_COL (sprite=0, type=sprite-sprite)
    BEQ no_hit          ; A = 0 → no collision
    LDA #$02
    STA $D020           ; red border = hit!
```

```
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; light blue border = clear
    JMP gameloop
```

> **See also:** `collision-demo` sample — green ball (sprite #0) vs. red cross (sprite #1).

---

9. Knowledge Base Links

Quick reference links available in the app under **Knowledge Base**:

Resource	URL
6502 Opcodes Reference	http://www.6502.org/tutorials/6502opcodes.html
C64 KERNAL Functions	https://sta.c64.org/cbm64krnfunc.html
C64 Memory Map	https://sta.c64.org/cbm64mem.html
C64 Color Codes	https://sta.c64.org/cbm64col.html
VIC-II Article	https://www.cebix.net/VIC-Article.txt
